15 · Distributed Training, GPU Scheduling and Tuning

> **Goal:** After this chapter you can (a) choose a parallelism strategy from model size and interconnect and estimate memory before launching, (b) make long training runs survive failures and preemption, and (c) schedule, tune and profile GPU workloads so utilisation turns into useful work.

Table of Contents

- 15.1 Parallelism Strategies: Data, Tensor, Pipeline and Sharded Training
- 15.2 Fault-Tolerant and Preemptible Training
- 15.3 GPU Cluster Scheduling, Queues and Utilisation
- 15.4 Hyperparameter Optimisation and AutoML at Scale
- 15.5 Training Throughput: Input Pipelines, Mixed Precision and Bottleneck Diagnosis

15.1 Parallelism Strategies: Data, Tensor, Pipeline and Sharded Training

Conceptual Deep Dive

Training memory has four parts: **weights, gradients, optimizer states and activations**. With mixed precision and Adam, a common accounting is 2 bytes (half-precision weights) + 2 (gradients) + 12 (fp32 master weights plus two moment buffers) = about 16 bytes per parameter before activations (rule of thumb). A 13B-parameter model therefore needs roughly 200 GB of model state, far beyond one GPU. Activations scale with batch size, sequence length and depth, and are the lever that gradient checkpointing pulls.

Data parallelism (DDP) replicates the model on every rank, gives each a slice of the batch, and all-reduces gradients each step. It is simple and scales well while the model fits. **Gradient accumulation** sums micro-batch gradients before one optimizer step, so effective batch = micro-batch x accumulation steps x ranks. Learning-rate scaling with batch size (linear scaling with warm-up) is a heuristic with a ceiling: beyond a critical batch size returns diminish and generalisation can worsen.

Sharded data parallelism removes the replication. ZeRO (Rajbhandari et al., 2020) stage 1 shards optimizer states, stage 2 adds gradients, stage 3 adds parameters; PyTorch FSDP full-sharding is the analogue of stage 3. Memory per GPU falls roughly with the rank count, at the price of extra communication (all-gather of parameters before each layer, reduce-scatter of gradients; the ZeRO paper puts stage 3 near 1.5x the baseline volume, verify). Use it when the replica does not fit or leaves no room for activations.

Tensor parallelism splits matrix multiplications inside a layer across GPUs (Megatron-style, Shoeybi et al., 2019); it needs an all-reduce per layer, so it belongs on fast intra-node links such as NVLink. **Pipeline parallelism** splits depth into stages and feeds micro-batches; the bubble fraction is roughly (p-1)/(m+p-1) for p stages and m micro-batches, so more micro-batches shrink idle time. **3D parallelism** combines data, tensor and pipeline axes, mapping tensor parallelism inside a node and pipeline across nodes.

Choosing a strategy: fits one GPU with room, use DDP; fits only when sharded, FSDP/ZeRO within a node first; larger or slow interconnect, add tensor parallelism inside the node and pipeline across nodes. Define **scaling**

efficiency as T1 / (N x TN) and measure it; adding GPUs stops helping when communication outgrows compute, the global batch cannot grow, or stragglers dominate. **Gradient checkpointing** recomputes activations in the backward pass (about one extra forward, rule of thumb) to save memory; **offload** to CPU or NVMe trades PCIe bandwidth for capacity.

Edge cases. A very large global batch can hurt generalisation. Stragglers slow every synchronous step because all ranks wait. Non-divisible batch sizes leave the last shard smaller, so averaging per-rank means is biased unless weighted by sample count (shown below).

Cross-Connection Scenarios

- Fitting these jobs onto shared GPUs and gang scheduling: 9.16 and 15.3.
- Surviving failures during long distributed runs: 15.2.
- Measuring data-loading and communication bottlenecks before adding GPUs: 15.5.
- Data-parallel sampling and shuffling interact with sample-exact resume in 15.2.

Production Anti-Patterns

- Adding GPUs without measuring scaling efficiency, so cost doubles while step time barely moves.
- Ignoring interconnect bandwidth: tensor parallelism across slow links turns every layer into a network wait.
- Different dataloader seeds or shard assignments across ranks, causing duplicated or missing data per epoch and silently skewed training.

Minimal Working Code

The first block estimates per-GPU training memory by ZeRO stage; all figures are rough estimates with assumptions as parameters.

```
def train_mem_gb(params_b, n_gpus=1, zero=0, act_gb_per_b=1.5,
                 bytes_w=2, bytes_g=2, bytes_opt=12):
    """Rough per-GPU GB (1e9 bytes). Billions of params x bytes/param = GB.
    act_gb_per_b is an ASSUMPTION; real activations depend on batch, sequence, checkpointing.""
"
    w = bytes_w * params_b / (n_gpus if zero >= 3 else 1)
    g = bytes_g * params_b / (n_gpus if zero >= 2 else 1)
    o = bytes_opt * params_b / (n_gpus if zero >= 1 else 1)
    return w + g + o + act_gb_per_b * params_b

if __name__ == "__main__":
    gpu_gb = 80                             # assumed 80 GB-class device
    print("params  gpus  zero  est_GB  fits")
    for size in (1, 7, 13):
        for zero in (0, 1, 2, 3):
            m = train_mem_gb(size, n_gpus=8, zero=zero)
            print(f"{size:>4}B  {8:>4}  {zero:>4}  {m:6.1f}  {'yes' if m <= gpu_gb else
'no'}")
```

The second block verifies that data-parallel gradient averaging and gradient accumulation match single-worker large-batch SGD, and shows the uneven-shard pitfall.

```
import numpy as np
rng = np.random.default_rng(0)
X = rng.normal(size=(64, 5))
y = X @ np.arange(1, 6.0) + 0.1 * rng.normal(size=64)

def grad(w, Xb, yb):
    return 2 * Xb.T @ (Xb @ w - yb) / len(yb)

def run(mode, steps=20, lr=0.05, k=4):
    w = np.zeros(5)
```

```
    for _ in range(steps):
        if mode == "single":
            g = grad(w, X, y)
        elif mode == "dp":                              # all-reduce mean over k ranks
            g = np.mean([grad(w, a, b) for a, b in zip(np.split(X, k), np.split(y, k))], axis=0)
        elif mode == "accum":                           # one rank, k micro-batches
            g = sum(grad(w, a, b) / k for a, b in zip(np.split(X, k), np.split(y, k)))
        elif mode == "uneven_naive":                    # 64 samples over 3 ranks: 22/21/
21
            g = np.mean([grad(w, a, b) for a, b in zip(np.array_split(X, 3), np.array_split(y, 3))], axis
=0)
        elif mode == "uneven_weighted":
            parts = list(zip(np.array_split(X, 3), np.array_split(y, 3)))
            g = sum(len(b) * grad(w, a, b) for a, b in parts) / 64
        w = w - lr * g
    return w

if __name__ == "__main__":
    ref = run("single")
    for m in ("dp", "accum", "uneven_naive", "uneven_weighted"):
        print(f"{m:16s} max|w - w_single| = {np.abs(run(m) - ref).max():.2e}")
```

Interview Angle

Q: A 13B-parameter model does not fit on one GPU. Walk through your options. **A:** Estimate state at about 16 bytes per parameter, so roughly 200 GB before activations. Shard with FSDP or ZeRO-3 across a node, add gradient checkpointing and mixed precision, then consider tensor parallelism within the node and pipeline across nodes if still tight or slow. For fine-tuning, consider parameter-efficient methods. Trade-off: more sharding cuts memory but adds communication. Measurable check: peak memory per GPU, tokens per second per GPU, and scaling efficiency versus the smaller setup.

Q: Why does scaling from 8 to 64 GPUs often give less than 8x speedup? **A:** Communication grows relative to compute across slower inter-node links, the global batch must grow (hitting the critical batch size and needing re-tuned learning rates), stragglers gate every synchronous step, and input pipelines may not keep up. Fixes: overlap communication with compute, larger micro-batches, topology-aware placement, faster storage. Measurable check: scaling efficiency (throughput at 64 GPUs divided by throughput at 8, divided by 8), plus a profile showing time in collectives and data wait.

15.2 Fault-Tolerant and Preemptible Training

Conceptual Deep Dive

Long runs fail. The failure types are node loss and spot preemption, network or NCCL hangs, out-of-memory errors, corrupt data, and silent numerical problems (NaN or inf loss, gradient explosions). The design goal is bounded lost work: after any failure, at most one checkpoint interval of progress disappears and the restart needs no manual repair.

A **checkpoint** must contain everything that determines the future: model weights, optimizer state (momentum and moments; without it Adam restarts "cold" and the loss spikes), learning-rate scheduler state, step and epoch counters, RNG states, mixed-precision scaler state and dataloader position. Writes must be **atomic**: write to a temporary file in the same directory, fsync, then rename, so a process killed mid-write never leaves a half-written "latest". Validate with a checksum and a load test, keep the last few checkpoints, and let the loader fall back to the newest valid one. For sharded models each rank writes its shard; commit a manifest only after all shards land.

Choose frequency by balancing overhead against expected lost work. The Young/Daly first-order rule puts the optimal interval near sqrt(2 x checkpoint_cost x mean_time_between_failures); treat it as a starting point and tune against measured costs. Asynchronous checkpointing hides write time.

Resumability means sample-exact continuation: derive data order from (seed, epoch) so position alone reconstructs the stream, restore RNG states, and make job submission idempotent (the job id maps to a checkpoint directory, so a retry resumes rather than restarts). **Elastic training** (torchrun elastic or Ray Train style restart policies; check current docs) lets a job continue when membership changes, but a gang restart still costs the time to reload state and re-form collectives.

On **spot or preemptible** capacity, handle the termination notice (a SIGTERM with a short grace period, provider specific): checkpoint immediately, exit cleanly, and let the controller requeue. Mixed pools (a stable on-demand base plus spot for the rest) cap the downside. Spot saves money only if wasted work stays below the discount (16.4 models the break-even).

Monitoring training health catches the silent failures: loss spikes, gradient norm, throughput per step, GPU utilisation and memory. Utilisation alone is a poor hang detector: a rank stuck in a collective can show busy GPUs. Use a progress heartbeat (step counter advancing) and a watchdog with a timeout on collectives (check the framework's timeout settings).

Edge cases. A checkpoint written while the process dies is corrupt unless writes are atomic. Resuming with a different data order changes results and makes divergence impossible to debug. A hang with all GPUs at 100% utilisation and no progress is the signature of a stuck collective.

Cross-Connection Scenarios

- Spot GPU pools and graceful drain behaviour in the scheduler: 9.16.
- Queue policy decides how often preemption happens: 15.3.
- Cost break-even of spot versus on-demand with re-run waste: 16.4.
- Experiment and artifact versioning of checkpoints: Module 03 (legacy).

Production Anti-Patterns

- Checkpointing only model weights, so every resume restarts the optimizer cold and the loss spikes or the run diverges.
- No watchdog for stuck collectives: a hung job burns GPU-hours for days while dashboards show 100% utilisation.
- Training on spot with no resume test, discovering at the first preemption that the checkpoint cannot be loaded.

Minimal Working Code

The first block writes checkpoints atomically with a checksum and loads the newest valid one, falling back past a corrupted file.

```
import hashlib, json, os, tempfile
from pathlib import Path

def save(dirp, step, state):
    body = json.dumps(state, sort_keys=True)
    blob = json.dumps({"sha256": hashlib.sha256(body.encode()).hexdigest(), "body": b
ody})
    final = Path(dirp) / f"ckpt_{step:08d}.json"
    fd, tmp = tempfile.mkstemp(dir=dirp, suffix=".tmp")
    with os.fdopen(fd, "w") as f:
        f.write(blob); f.flush(); os.fsync(f.fileno())
    os.replace(tmp, final)                      # atomic on the same filesystem
    return final

def load_latest(dirp):
    for p in sorted(Path(dirp).glob("ckpt_*.json"), reverse=True):
        try:
            rec = json.loads(p.read_text())
            if hashlib.sha256(rec["body"].encode()).hexdigest() == rec["sha256"]:
                return p.name, json.loads(rec["body"])
        except (ValueError, KeyError):
            pass                                # corrupt or truncated: try the previous one
    return None, None

if __name__ == "__main__":
    with tempfile.TemporaryDirectory() as d:
        for step in (100, 200, 300):
            save(d, step, {"step": step, "w": [0.1 * step]})
        newest = Path(d) / "ckpt_00000300.json"
        newest.write_text(newest.read_text()[:20])      # simulate a crash mid-write
        print(load_latest(d))                           # falls back to step 200
```

The second block is a toy training loop that checkpoints on SIGTERM and resumes sample-exactly, verified against an uninterrupted run.

```
import json, random, signal
N, BATCH, TOTAL = 64, 8, 100
DATA = [(i / N, 3 * (i / N) + 0.5) for i in range(N)]
```

```
class Trainer:
    def __init__(self, state=None, seed=0):
        self.seed, self.stop = seed, False
        self.s = state or dict(step=0, w=0.0, b=0.0, rng=None)
    def order(self, epoch):
        idx = list(range(N)); random.Random(self.seed * 1000 + epoch).shuffle(idx)
        return idx
    def run(self, kill_at=None):
        signal.signal(signal.SIGTERM, lambda *_: setattr(self, "stop", True))
        rng, s = random.Random(self.seed + 1), self.s
        if s["rng"]: rng.setstate((s["rng"][0], tuple(s["rng"][1]), s["
;rng"][2]))
        while s["step"] < TOTAL and not self.stop:
            epoch, i = divmod(s["step"], N // BATCH)
            ids, noise = self.order(epoch)[i * BATCH:(i + 1) * BATCH], rng.gauss(0, 0.01)
            gw = gb = 0.0
            for k in ids:
                x, y = DATA[k]; e = s["w"] * x + s["b"] - y + noise
                gw += 2 * e * x / BATCH; gb += 2 * e / BATCH
            s["w"] -= 0.5 * gw; s["b"] -= 0.5 * gb; s["step"] += 1
            if kill_at == s["step"]: signal.raise_signal(signal.SIGTERM)
        st = rng.getstate(); s["rng"] = [st[0], list(st[1]), st[2]]
        return json.dumps(s)                          # checkpoint payload

if __name__ == "__main__":
    full = json.loads(Trainer().run())
    part = Trainer().run(kill_at=37)                  # SIGTERM at step 37 -> checkpoint
    resumed = json.loads(Trainer(json.loads(part)).run())
    print("stopped at", json.loads(part)["step"], "| identical to uninterrupted:
", resumed == full)
```

Interview Angle

Q: Design a 3-day training run on spot instances. What guarantees progress? **A:** Frequent atomic checkpoints of full state (weights, optimizer, scheduler, RNG, data position) to durable storage, a SIGTERM handler that saves on the termination notice, deterministic data order for sample-exact resume, idempotent job ids, and automatic requeue. Keep a small on-demand base if the budget allows. Trade-off: checkpoint overhead versus lost work, tuned with the Young/Daly estimate. Measurable check: kill-and-resume drills showing identical loss curves and wasted work under a target percentage.

Q: Loss became NaN at step 50k. How do you debug and recover? **A:** Stop and roll back to the last good checkpoint rather than patching forward. Inspect gradient norm and loss history before the spike, check the batch that triggered it for corrupt or extreme data, and test mixed-precision overflow (loss-scale), learning-rate warm-up and normalisation epsilon. Mitigate with gradient clipping, lower learning rate, skipping the bad batch, and NaN checks in the loop. Measurable check: the run passes the failing step in a replay with the fix, and grad-norm alerts fire earlier next time.

15.3 GPU Cluster Scheduling, Queues and Utilisation

Conceptual Deep Dive

Different workload classes need different rules. **Interactive notebooks** want fast start and small slices but waste idle GPUs; **batch training** is throughput-oriented, large and preemptible if checkpointed (15.2); **HPO sweeps** (15.4) are many small, restartable trials; **online inference** needs reserved capacity and the highest priority. One FIFO queue for all of them guarantees someone is unhappy.

Scheduler options, at concept level (check current docs for features): Kubernetes with an admission and queueing layer such as Kueue or a batch scheduler such as Volcano (gang scheduling, queues, quotas); Slurm, the long-standing HPC scheduler with strong partition, fair-share and backfill semantics; Ray's own cluster scheduling for Ray workloads; and managed training services that hide the scheduler at the cost of control. The choice is driven by what the organisation already runs, how much multi-tenancy it needs, and how much job heterogeneity exists.

Policies. **FIFO** is simple but suffers head-of-line blocking: a job needing eight GPUs at the head idles seven GPUs while waiting for the eighth. **Priority with preemption** lets urgent work jump the queue at the cost of wasted partial work. **Fair-share** orders by recent usage versus entitlement, so a team that has used little goes first; it can penalise bursty teams whose usage is spiky.

Backfilling lets small jobs run in the gaps as long as they do not delay the reserved start of the blocked head job. **Quota borrowing and reclaim** lets teams use idle capacity beyond their quota and take it back by preemption when the owner returns (9.16 shows the mechanics).

Measure at the right level. **Allocated** GPUs are reserved; **busy** GPUs show kernel activity; **useful** work advances the objective, which MFU (model FLOPs utilisation, achieved model FLOPs divided by peak) approximates for training. Add queue wait time (by job size and team), job failure and restart rates, and the idle share of reserved capacity. "Utilisation is 55% while jobs wait hours" usually means fragmentation, gang constraints, quotas or idle allocations, not a shortage of GPUs.

Capacity planning forecasts demand from job traces: hourly concurrent GPU demand, its percentiles and seasonality (nights, weekends, release deadlines). Reserved or committed capacity covers the steady base; spot or on-demand covers bursts; keep headroom proportional to the wait-time target. Developer experience reduces waste at the source: sizing guidance, templates with sane defaults, and automatic idle shutdown.

Edge cases. Large jobs starve behind a stream of small ones unless the scheduler reserves for the head job (backfill with a reservation, or aging). Fair-share with a short decay window punishes teams that run in bursts. Reserved capacity sits idle at night unless low-priority preemptible work backfills it.

Cross-Connection Scenarios

- Fragmentation and gang scheduling mechanics on Kubernetes: 9.16.
- Spot and preemption handling inside the job: 15.2.
- Quota allocation and policy guardrails per team: 16.2; cost attribution by GPU-hours used: 16.3.
- Parallelism choice determines how many GPUs a job must receive at once: 15.1.

Production Anti-Patterns

- First-come-first-served only, so a single 8-GPU job at the head blocks a queue of small jobs that could run now.
- No visibility into queue wait, so complaints are anecdotes and nobody can tell scarcity from poor scheduling.
- Static reservations per team, so idle quota is stranded while another team queues.

Minimal Working Code

The first block compares FIFO, priority and EASY-style backfill on a synthetic job trace, reporting mean wait and utilisation.

```
import random

def trace(n=40, seed=5):
    r, t, out = random.Random(seed), 0, []
    for i in range(n):
        t += r.choice([0, 1, 1, 2, 3])
        out.append(dict(id=i, arr=t, g=r.choice([1, 1, 2, 4, 8]), dur=r.randint(1, 8), prio=r.choice([0,
0, 1])))
    return out

def simulate(jobs, policy, cap=8):
    arrivals, queue, running, waits, busy, t = sorted(jobs, key=lambda j: j["arr"]), [], [], []
, 0, 0
    def start(j, free):
        running.append((t + j["dur"], j["g"])); waits.append((j["prio"], t
- j["arr"])); return free - j["g"]
    while arrivals or queue or running:
        running = [(e, g) for e, g in running if e > t]
        while arrivals and arrivals[0]["arr"] <= t: queue.append(arrivals.pop(0))
        if policy == "priority": queue.sort(key=lambda j: (-j["prio"], j["arr&qu
ot;]))
        free = cap - sum(g for _, g in running)
        while queue and queue[0]["g"] <= free: free = start(queue.pop(0), free)
        if policy == "backfill" and queue:
            acc, shadow = free, t                    # earliest time the head job can start
            for e, g in sorted(running):
                acc += g; shadow = e
                if acc >= queue[0]["g"]: break
            for j in list(queue[1:]):
```

```
            if j["g"] <= free and t + j["dur"] <= shadow:
                queue.remove(j); free = start(j, free)
        busy += cap - free; t += 1
    hi = [w for p, w in waits if p == 1]
    return sum(w for _, w in waits) / len(waits), sum(hi) / len(hi), busy / (cap * t)

if __name__ == "__main__":
    jobs = trace()
    for p in ("fifo", "priority", "backfill"):
        w, hi, u = simulate([dict(j) for j in jobs], p)
        print(f"{p:9s} mean_wait={w:5.2f}h high_prio_wait={hi:5.2f}h utilisation={u:.2f}")
```

The second block proposes reserved versus spot capacity from an hourly demand trace using percentile math and relative (abstract) prices.

```
import numpy as np

def plan(demand, wait_pct=95, reserved_price=0.6, spot_price=1.0, spot_waste=1.15):
    """Prices are relative, abstract units. Total capacity covers demand in wait_pct% of h
ours."""
    total = int(np.ceil(np.percentile(demand, wait_pct)))
    best = None
    for R in range(total + 1):
        served = np.minimum(demand, total)
        spot_hours = np.clip(served - R, 0, None).sum()
        cost = R * len(demand) * reserved_price + spot_hours * spot_price * spot_waste
        util = np.minimum(demand, R).sum() / (R * len(demand)) if R else 0.0
        if best is None or cost < best[0]:
            best = (cost, R, util)
    cost, R, util = best
    return dict(total=total, reserved=R, spot=total - R, reserved_util=round(float(util), 2), cost=round(
float(cost)))

if __name__ == "__main__":
    rng = np.random.default_rng(0)
    hours = np.arange(24 * 28)
    demand = np.round(20 + 8 * np.sin(2 * np.pi * hours / 24) + rng.gamma(2, 2, hours.size)).astype(int)
    print("demand p50/p95/p99:", [int(np.percentile(demand, q)) for q in (50, 95, 99)])
    for pct in (90, 95, 99):
        print(f"target {pct}% of hours without queueing ->", plan(demand, pct))
```

Interview Angle

Q: Researchers complain jobs wait hours while utilisation shows 55%. Diagnose. **A:** Separate allocated from busy from useful. Look at largest free block versus job sizes (fragmentation), head-of-line blocking, gangs that cannot fit, exhausted team quotas, and notebooks holding idle GPUs. Check wait time by job size. Fixes: backfill, best-fit packing, idle reclamation, quota borrowing. Trade-off: preemption wastes partial work. Measurable check: queue wait p90 by size class and busy-GPU percentage before and after, with job restart rate watched as the side effect.

Q: Design the queue and quota policy for three teams sharing 64 GPUs. **A:** Guaranteed quotas that sum to roughly 80-90% of capacity (rule of thumb), borrowing of idle GPUs by anyone, reclaim by preempting borrowers (checkpointed jobs only), priority classes for inference above training above sweeps, backfill with a reservation for large jobs, and aging so nothing starves. Trade-off: guarantees reduce flexibility, borrowing recovers it. Measurable check: per-team wait percentiles, borrowed GPU-hours, preemption rate and utilisation.

15.4 Hyperparameter Optimisation and AutoML at Scale

Conceptual Deep Dive

Search strategies range from **grid** (exhaustive over a lattice) and **random** to **Bayesian** methods (TPE or Gaussian-process surrogates that propose points where improvement looks likely) and **population-based training** (Jaderberg et al., 2017), which mutates hyperparameters while training. Random search beats grid in high dimensions because usually only a few

hyperparameters matter: a grid repeats the same few values of the important one, while random draws give a fresh value of it on every trial (Bergstra and Bengio, 2012).

Multi-fidelity methods spend little on bad configurations. Successive halving (Jamieson and Talwalkar, 2016) trains many configurations on a small budget, keeps the top fraction (1/eta) and repeats with eta-times the budget; Hyperband (Li et al., 2017) hedges across several starting budgets because the right starting point is unknown; ASHA (Li et al., 2018) makes promotion asynchronous so workers never idle. Hazards: rankings at low fidelity may not match final rankings, so slow starters (long warm-up, large models whose gains arrive late) are killed early. Mitigate with a sensible minimum budget, learning-rate warm-up inside the budget, and a sanity run of the winner versus a few full-budget baselines.

Distributed execution treats each trial as a job: trial parallelism, resource-aware placement (small trials packed, big ones gang scheduled; see 9.16 and 15.3), checkpointed trials so pause, resume and preemption work (15.2), and a reproducible seed per trial derived from the trial id and logged. A shared cache that leaks information across folds invalidates comparisons, so isolate caches by fold.

Honest evaluation. The maximum of many noisy validation scores is optimistically biased (winner's curse). Keep a final test set untouched until the single end-of-search evaluation, use nested validation when data is small, re-run the top few configurations with different seeds, and report the number of trials alongside the score. A noisy objective makes Bayesian optimisation chase noise; average repeated evaluations or model the noise.

AutoML helps with baselines, feature and pipeline search, and quickly bounding what is achievable. Risks: target leakage inside automated feature generation, opaque pipelines that are hard to review, runaway cost, and ungoverned promotion. Auto-selected models still pass registry gates and model cards (Module 04 (legacy)), with the search space, seeds and budget logged.

Search-space design matters as much as the algorithm: sample learning rates and regularisation on a log scale, encode conditional parameters (momentum only for SGD), and turn latency or size budgets from Module 06 (legacy) into constraints or penalties so the winner is deployable. Bound cost with a stopping rule: a maximum number of trials, wall-clock or GPU-hour budget, and a plateau criterion.

Cross-Connection Scenarios

- Preemptible, restartable trials rely on checkpointing from 15.2.
- Trial scheduling and quotas for sweeps: 15.3 and 16.2; budget guardrails for sweeps: 16.3.
- Experiment tracking of every trial and its seed: Module 03 (legacy).
- Cheaper proxy models for HPO are a cost lever in 16.4.

Production Anti-Patterns

- Tuning on the test set (or reusing it after every change), so the reported metric is an optimistic artefact of the search.
- A huge search with no stopping rule or budget, consuming GPU-weeks for gains inside the noise.
- Choosing the best trial from a single run without a repeat, which promotes seed luck.

Minimal Working Code

The first block compares successive halving with random search at equal total budget on a synthetic noisy objective.

```
import numpy as np

def loss(q, s, r, rng):
    """Config quality q (lower is better) + slow-start term s/sqrt(r) + evaluation noise.&
quot;""
    return q + s / np.sqrt(r) + rng.normal(0, 0.05, np.shape(q))

def successive_halving(rng, n0=81, r0=1, eta=3):
    q, s = rng.uniform(0, 1, n0), rng.uniform(0.2, 1.0, n0)
    idx, r, budget = np.arange(n0), r0, 0
    while True:
        budget += len(idx) * r
        if len(idx) == 1: break
        keep = max(1, len(idx) // eta)
        idx = idx[np.argsort(loss(q[idx], s[idx], r, rng))[:keep]]
        r *= eta
    return q[idx[0]], budget, r
```

```
def random_search(rng, budget, r_full):
    k = budget // r_full
    q, s = rng.uniform(0, 1, k), rng.uniform(0.2, 1.0, k)
    return q[np.argmin(loss(q, s, r_full, rng))]

if __name__ == "__main__":
    rng = np.random.default_rng(0)
    sh, rs = [], []
    for _ in range(300):
        q, budget, r_full = successive_halving(rng)
        sh.append(q); rs.append(random_search(rng, budget, r_full))
    print(f"budget={budget} units, full-budget runs for random search={budget // r_full}")
    print(f"true quality of winner (lower is better): SH={np.mean(sh):.3f}  random={np.mean(rs):.3f}
")
```

The second block shows why random search beats grid when only one parameter matters.

```
import numpy as np

def f(x1, x2):
    return (x1 - 0.37) ** 2 + 0.01 * (x2 - 0.5) ** 2        # only x1 really matters

if __name__ == "__main__":
    g = np.linspace(0, 1, 3)
    grid = [(a, b) for a in g for b in g]                    # 9 trials
    best_grid = min(f(a, b) for a, b in grid)
    rng = np.random.default_rng(0)
    best_rand = []
    for _ in range(500):
        pts = rng.uniform(0, 1, size=(9, 2))                 # 9 trials
        best_rand.append(f(pts[:, 0], pts[:, 1]).min())
    print("distinct x1 values tried: grid =", len({round(a, 6) for a, _ in grid}), " rando
m = 9")
    print(f"best loss with 9 trials: grid={best_grid:.4f}  random(mean of 500 runs)={np.mean(best_ra
nd):.4f}")
```

Interview Angle

Q: How do you set up HPO so the reported test metric is honest? **A:** Split data into train, validation for search, and a final test set touched once. Use nested or cross-validation for small data, fix and log seeds per trial, re-run the top configurations on several seeds and report mean and spread, and publish the number of trials. Trade-off: honest protocols cost extra compute and data. Measurable check: the gap between validation-selected score and final test score; a consistently large gap means the search is overfitting.

Q: When is Bayesian optimisation not worth it? **A:** When trials are cheap (random search with many trials suffices), the objective is very noisy, the space is high-dimensional and mostly categorical, or parallelism is large and sequential proposals idle workers. It shines for expensive trials with few continuous parameters. Trade-off: model overhead and tuning versus fewer trials. Measurable check: best-so-far curve per GPU-hour versus a random-search baseline run with the same budget.

15.5 Training Throughput: Input Pipelines, Mixed Precision and Bottleneck Diagnosis

Conceptual Deep Dive

A training step spends time in data loading, host-to-device transfer, GPU compute, inter-GPU communication and periodic checkpointing. **Profile before tuning**: an optimisation of a non-bottleneck stage changes nothing. Use a framework profiler or a system timeline tool (check current docs) and read the timeline for gaps between kernels, which mean the GPU waits on the CPU, the data loader or the network.

The **input pipeline** is the most common culprit. Use parallel loader workers, prefetch the next batches while the current one computes, pin host memory for faster transfer, cache decoded or preprocessed data when augmentation is deterministic, and

prefer sequential sharded formats over many small files (9.17). Python-bound augmentation (per-sample interpreted code) caps throughput regardless of GPU speed; vectorise it or move it to the GPU. Diagnose GPU idle gaps by comparing step time with and without real data (synthetic batches): if synthetic is much faster, the pipeline is the bottleneck.

Mixed precision lowers memory and raises throughput. FP16 has a narrow range, so loss scaling prevents small gradients from underflowing; BF16 has fp32's range and usually needs no scaler. Keep numerically sensitive operations (reductions, softmax, normalisation statistics) in higher precision, and watch for NaN or inf. Compilers and kernel fusion (torch.compile style; check current docs) cut launch overhead and memory traffic but add compile time and graph-break pitfalls.

Batching. Larger micro-batches raise utilisation until memory or generalisation limits bite; gradient accumulation gets the effective batch without the memory. For variable-length sequences, random batching pads every sequence to the longest in its batch, wasting compute; **length bucketing** groups similar lengths (with shuffling between batches to avoid ordering bias).

Headline metrics. Utilisation is not efficiency: a GPU can be 100% busy running inefficient kernels. Use **MFU** (achieved model FLOPs divided by hardware peak) or throughput per dollar (samples or tokens per second per cost unit) as the target, and benchmark properly: discard warm-up and compilation steps, measure steady state over enough steps, synchronise the device before reading timers because GPU execution is asynchronous, repeat runs and report median and spread. Add a **regression guard** in CI: a short throughput test on a fixed hardware class with a tolerance band, failing the build when a change slows training beyond it.

Edge cases. Loader workers beyond the available CPU cores oversubscribe and slow everything (also contending with the main process). FP16 can underflow small gradients without a scaler. A benchmark that includes warm-up or compilation time overstates the cost of a faster configuration.

Cross-Connection Scenarios

- Distributed communication overhead versus data wait: 15.1.
- Storage format and shard layout for fast reads: 9.17.
- Throughput per dollar feeds cost optimisation and unit economics: 16.3 and 16.4.
- Serving-side latency and batching are in Module 06 (legacy).

Production Anti-Patterns

- Optimising the model while the GPU waits on data, so weeks of kernel tuning produce zero speedup.
- Reporting speedups from a single noisy run, which later cannot be reproduced.
- No throughput baseline, so regressions from code or dependency changes are discovered via the invoice.

Minimal Working Code

The first block times pipeline stages with a context manager and names the bottleneck, using simulated sleeps in place of real work.

```
import time
from collections import defaultdict
from contextlib import contextmanager

class StageTimer:
    def __init__(self): self.t = defaultdict(float)
    @contextmanager
    def stage(self, name):
        start = time.perf_counter()
        try: yield
        finally: self.t[name] += time.perf_counter() - start     # real code: sync device first
    def report(self):
        total = sum(self.t.values())
        for k, v in sorted(self.t.items(), key=lambda kv: -kv[1]):
            print(f"{k:9s} {1000 * v:7.1f} ms  {100 * v / total:5.1f}%")
        return max(self.t, key=self.t.get)

def work(ms):                    # precise busy-wait: time.sleep granularity is coarse on some platform
s
    end = time.perf_counter() + ms / 1000
    while time.perf_counter() < end:
        pass
```

```
if __name__ == "__main__":
    timer = StageTimer()
    for _ in range(15):
        with timer.stage("data"): work(6)               # simulated slow loader
        with timer.stage("transfer"): work(1)
        with timer.stage("compute"): work(3)
    bottleneck = timer.report()
    print("bottleneck:", bottleneck, "| GPU busy fraction ~", round(timer.t["com
pute"] / sum(timer.t.values()), 2))
    assert bottleneck == "data"
```

The second block quantifies padding waste for random batching versus length-bucketed batching on a skewed length distribution.

```
import numpy as np

def padding_waste(lengths, batches):
    padded = sum(max(lengths[i] for i in b) * len(b) for b in batches)
    return 1 - lengths.sum() / padded

def random_batches(n, bs, rng):
    idx = rng.permutation(n)
    return [idx[i:i + bs] for i in range(0, n, bs)]

def bucketed_batches(lengths, bs, rng):
    order = np.argsort(lengths)
    batches = [order[i:i + bs] for i in range(0, len(order), bs)]
    rng.shuffle(batches)                            # shuffle batch order, not contents
    return batches

if __name__ == "__main__":
    rng = np.random.default_rng(0)
    lengths = np.clip(rng.lognormal(mean=4.0, sigma=0.7, size=4096), 8, 512).astype(int)
    for bs in (16, 64):
        r = padding_waste(lengths, random_batches(len(lengths), bs, rng))
        b = padding_waste(lengths, bucketed_batches(lengths, bs, rng))
        print(f"batch={bs:3d} padding waste: random={r:.1%} bucketed={b:.1%}")
```

Interview Angle

Q: GPU utilisation is 35% during training. What do you check, in order? **A:** First profile one step to see where time goes. Then compare real versus synthetic batches to test the input pipeline; check loader workers, prefetch, pinned memory, file format and CPU saturation. Next check host-to-device transfer and small-kernel launch overhead, then communication in multi-GPU runs, then batch size and precision. Trade-off: larger batches and caching cost memory. Measurable check: step time and MFU after each single change, never several at once.

Q: How do you benchmark a training change fairly? **A:** Same hardware, data and seeds; discard warm-up and compilation steps; synchronise before timing; run enough steady-state steps and repeat several times; report median and spread plus throughput per dollar and final quality, since a faster run that converges worse is no win. Trade-off: rigorous runs cost GPU time, so keep a short CI benchmark and a periodic long one. Measurable check: the speedup exceeds run-to-run variation, for example a gap larger than the observed spread.